

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ «Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Прикладная информатика

О Т Ч Е Т
Отчёт по курсовой работе дисциплины Мобильная
разработка (Android и IOs)

Обучающийся:
Шалунов Андрей Ильич, К3440

Преподаватель:
Шатинский Григорий Сергеевич

Санкт-Петербург
2026

АННОТАЦИЯ

Курсовая работа посвящена созданию учебного Android-приложения, которое имитирует базовый функционал мессенджера. Пользователь может просматривать ленту сообщений, редактировать профиль и изменять настройки приложения. Разработка велась поэтапно через четыре лабораторные работы. Сначала был создан каркас интерфейса и навигация между экранами. Затем в проект была внедрена архитектура MVVM и выполнено управление состоянием экранов через ViewModel и LiveData. После этого добавлена загрузка данных из публичного REST-API и сохранение сообщений в локальную базу Room, что обеспечило офлайн-доступ. На заключительном этапе усовершенствован пользовательский интерфейс, реализованы лайки, фоновая синхронизация данных через WorkManager и система уведомлений.

В работе использованы язык Kotlin, среда разработки Android Studio, компоненты Android Jetpack (ViewModel, LiveData, Navigation, Room, WorkManager), а также библиотеки Retrofit и OkHttp для взаимодействия с сетью. Интерфейс построен на базе элементов Material Design. Разработанное приложение демонстрирует полный цикл работы с данными в мобильном клиенте: получение сообщений из сети, их кэширование в базе, отображение в списке и фоновое обновление с уведомлением пользователя.

СОДЕРЖАНИЕ

1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРОЕКТА.....	5
1.1 Назначение и функции приложения.....	5
1.2 Этапы выполнения.....	5
2 ТЕХНОЛОГИЧЕСКАЯ БАЗА ПРОЕКТА.....	6
2.1 Платформа и инструменты разработки.....	6
2.2 Архитектура MVVM.....	6
2.3 Используемые библиотеки.....	7
3 РЕАЛИЗАЦИЯ ЛАБОРАТОРНЫХ РАБОТ.....	8
3.1 Лабораторная работа №1. Интерфейс и навигация.....	8
3.2 Лабораторная работа №2. Управление состоянием через ViewModel.....	9
3.3 Лабораторная работа №3. Сеть и база данных.....	11
3.4 Лабораторная работа №4. Интерфейс, лайки, фоновые задачи и уведомления.....	13
4 АРХИТЕКТУРНЫЕ ОСОБЕННОСТИ.....	16
4.1 Структура пакетов.....	16
4.2 Единый источник данных и офлайн-режим.....	16
5 ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС.....	17
5.1 Лента сообщений.....	17
5.2 Профиль пользователя.....	18
5.3 Экран настроек.....	19
5.4 Уведомления и разрешения.....	20
6 ТЕСТИРОВАНИЕ.....	21

ВВЕДЕНИЕ

Мессенджеры относятся к числу наиболее востребованных мобильных приложений. Пользователи ожидают от них понятного интерфейса, устойчивой работы при нестабильном соединении и своевременных уведомлений. Поэтому даже учебный проект в формате «мессенджера» является удобной площадкой для отработки основных навыков Android-разработки: организации навигации, управления состоянием, работы с сетью и локальным хранилищем, а также реализации фоновых задач.

Целью курсовой работы является разработка учебного Android приложения мессенджера с использованием современного стека технологий и архитектурного подхода MVVM.

Для достижения цели последовательно решались следующие задачи. Необходимо было спроектировать основной интерфейс приложения и систему навигации между экранами. Затем требовалось внедрить ViewModel и LiveData, чтобы сохранять состояние профиля и настроек при повороте экрана. Следующим шагом стала интеграция с публичным REST API и сохранение полученных данных в базе Room, что дало возможность работе в офлайн-режиме. После этого нужно было добавить фоновое обновление данных через WorkManager и реализовать уведомления о новых сообщениях. Отдельной задачей была настройка визуального оформления интерфейса с использованием компонентов Material Design и поддержкой светлой и тёмной темы.

В отчете рассмотрены используемые технологии, структура проекта, реализация каждого этапа и результаты тестирования.

1 ОБЩАЯ ХАРАКТЕРИСТИКА ПРОЕКТА

1.1 Назначение и функции приложения

Разработанное приложение представляет собой учебный мессенджер. Пользователь получает возможность просматривать список сообщений, условно представляющих диалоги или новости. Каждое сообщение можно отметить как понравившееся. Существуют отдельные экраны профиля, где редактируются имя и статус пользователя, и настроек, где осуществляется переключение темы оформления.

Загрузка сообщений выполняется из открытого сервиса <https://jsonplaceholder.typicode.com/posts>, который возвращает тестовые данные в формате JSON. Этого достаточно, чтобы продемонстрировать работу с сетью без организации собственного сервера. Для устойчивой работы при отсутствии интернета сообщения сохраняются в локальной базе данных и подставляются из нее при невозможности обратиться к сети.

1.2 Этапы выполнения

Работа разбита на четыре логических этапа, совпадающих с лабораторными заданиями. В первой лабораторной создан каркас приложения: одна активность, несколько фрагментов и навигация на основе `BottomNavigationView` и `Navigation Component`. Во второй лабораторной добавлены классы `ViewModel` и реактивное обновление интерфейса через `LiveData`, что позволило организовать управление состоянием профиля и настроек. Третий этап посвящен интеграции с REST-API и библиотекой `Room`, а также организации офлайн-режима. Четвертая лабораторная расширяет функционал: на этом шаге были разработаны кастомный адаптер списка, лайки, фоновая синхронизация через `WorkManager`, система уведомлений и запрос разрешений при первом запуске.

2 ТЕХНОЛОГИЧЕСКАЯ БАЗА ПРОЕКТА

2.1 Платформа и инструменты разработки

Приложение ориентировано на платформу Android с минимальной версией API 24 (Android 7.0). В качестве языка программирования выбран Kotlin, который официально рекомендован Google для разработки под Android. Разработка велась в Android Studio, сборка проекта осуществлялась с помощью системы Gradle и файла `build.gradle.kts`, где подключены все необходимые зависимости.

2.2 Архитектура MVVM

В качестве архитектурного подхода применён паттерн Model-View-ViewModel. Представление (Activity и Fragment) отвечает только за отображение данных и передачу пользовательских действий. ViewModel хранит состояние экрана, вызывает методы репозитория и не зависит от конкретных классов фрагментов. Слой данных содержит доступ к сети и к локальной базе. Благодаря этому изменение интерфейса не требует переписывания логики хранения данных, а код становится более структурированным и удобным для расширения.

Связь между View и ViewModel построена на основе LiveData. Фрагменты подписываются на объекты LiveData и автоматически реагируют на изменения данных. На диаграмме показан поток данных:

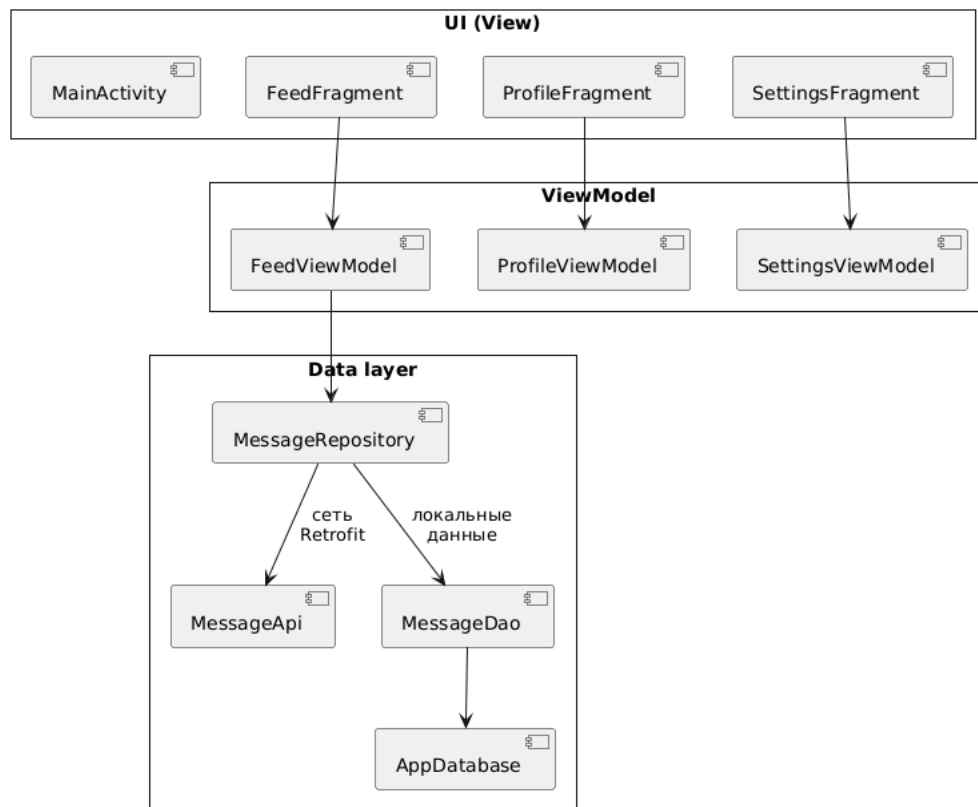


Рисунок 1 – Архитектура приложения по паттерну MVVM

2.3 Используемые библиотеки

В проекте применяются компоненты Android Jetpack: ViewModel и LiveData для управления состоянием, Navigation Component для навигации, Room для локального хранения, WorkManager для фоновых задач. Для сетевого взаимодействия выбрана связка Retrofit и OkHttp, которая позволяет описывать HTTP-запросы декларативно и выполнять их в корутинах. Пользовательский интерфейс реализован на основе Material Components: используются MaterialToolbar, FloatingActionButton, MaterialCardView, BottomNavigationView и другие элементы.

3 РЕАЛИЗАЦИЯ ЛАБОРАТОРНЫХ РАБОТ

3.1 Лабораторная работа №1. Интерфейс и навигация

На первом этапе сформирована базовая структура приложения. Создана главная активность MainActivity, в которой размещен NavHostFragment. Этот контейнер обслуживает переходы между фрагментами. Нижняя навигация реализована через BottomNavigationView. В ресурсах описан файл menu_bottom_nav.xml, где перечислены пункты «Лента», «Профиль» и «Настройки» с соответствующими иконками.

Связывание панели навигации с контроллером выполняется в MainActivity методом setupWithNavController, поэтому при нажатии на вкладку происходит переход к нужному фрагменту без дополнительного кода обработки.

Для анализа жизненного цикла фрагментов создан базовый класс LoggingFragment. В нём переопределены методы onCreate, onStart, onResume, onPause, onStop, onDestroyView и onDestroy, которые записывают сообщения в Logcat. Это позволило увидеть последовательность вызовов при открытии и закрытии экранов, а также при повороте устройства.

На Рисунке 2 показан внешний вид главного экрана с нижней панелью навигации.

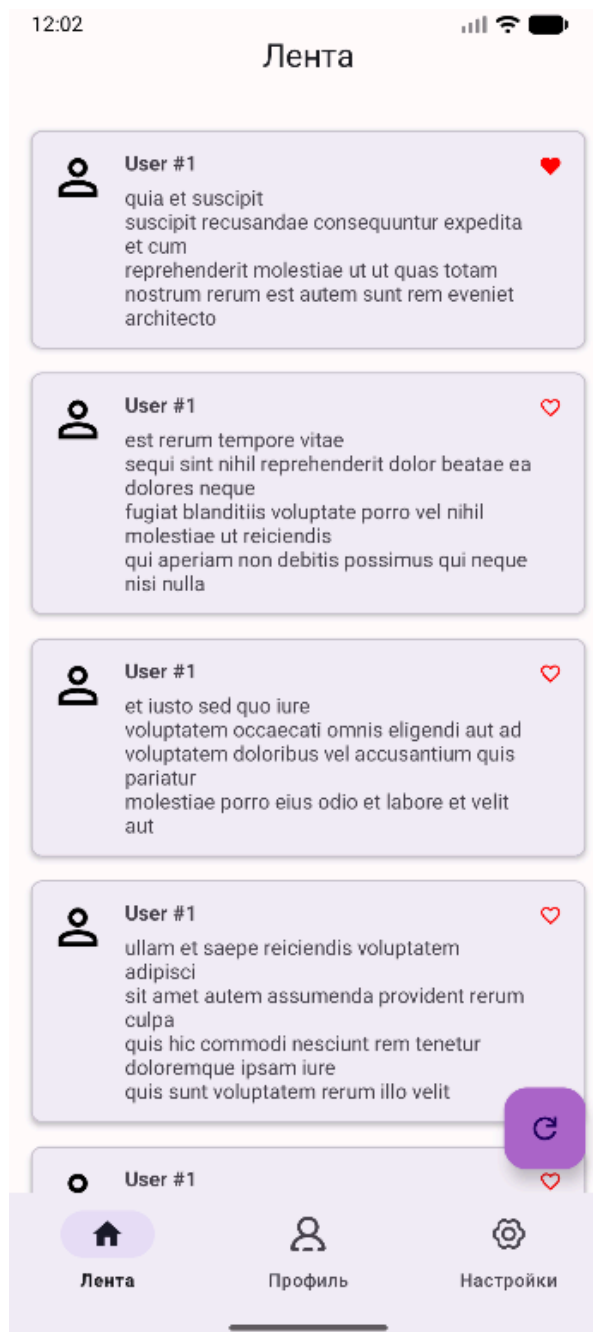


Рисунок 2 – Внешний вид главного экрана

3.2 Лабораторная работа №2. Управление состоянием через ViewModel

Вторая лабораторная работа посвящена архитектуре MVVM. Для экрана профиля создан класс `ProfileViewModel`. В нем с помощью `MutableLiveData` хранятся имя и статус пользователя. Эти данные инициализируются стандартными значениями, но могут быть изменены пользователем. В классе определены методы `changeName` и `changeStatus`,

которые обновляют соответствующие поля.

```
1 package com.mobile.lab1.viewmodel
2
3 > import ...
4
5 2 Usages
6
7
8 class ProfileViewModel : ViewModel() {
9
10     2 Usages
11     companion object {
12         2 Usages
13         private const val TAG = "ProfileViewModel"
14     }
15
16     2 Usages
17     private val _userName = MutableLiveData(value = "Андрей")
18     1 Usage
19     val userName: LiveData<String> = _userName
20
21     2 Usages
22     private val _userStatus = MutableLiveData(value = "Онлайн")
23     1 Usage
24     val userStatus: LiveData<String> = _userStatus
25
26     init {
27         Log.d(TAG, msg = "init: ProfileViewModel created")
28     }
29
30     1 Usage
31     fun updateName(newName: String) {
32         _userName.value = newName
33     }
34
35     1 Usage
36     fun updateStatus(newStatus: String) {
37         _userStatus.value = newStatus
38     }
39
40     override fun onCleared() {
```

Рисунок 3 – фрагмент ProfileViewModel

Фрагмент профиля ProfileFragment получает экземпляр ViewModel через делегат activityViewModels(), чтобы один и тот же объект использовался при пересоздании фрагмента и был общим для нескольких экранов при необходимости. Во фрагменте установлены наблюдатели за

LiveData, которые обновляют содержимое полей EditText только в том случае, если текст действительно изменился. Обратное направление связи реализовано с помощью слушателей doOnTextChanged, которые передают введенные значения обратно в ViewModel.

Аналогичная идея применена в классе SettingsViewModel, который хранит состояние переключателя темы. При изменении значения вызывается AppCompatActivity.setDefaultNightMode, а информация о текущем режиме доступна для подписчиков через LiveData.

Таким образом, логика хранения и изменения данных вынесена из фрагментов во ViewModel. При повороте устройства состояние профиля и настроек сохраняется, что соответствует требованиям второй лабораторной работы.

3.3 Лабораторная работа №3. Сеть и база данных

Третий этап реализует получение сообщений из удаленного сервиса и их сохранение в базе Room. В качестве источника данных используется сайт JSONPlaceholder. Описание API выполнено в интерфейсе MessageApi с использованием Retrofit. Основным методом является getMessages, который возвращает список DTO-объектов MessageDto.

```
1 package com.mobile.lab1.data
2
3 import retrofit2.http.GET
4
5 3 Usages
6 interface MessageApi {
7     1 Usage
8     @GET(value = "posts")
9     suspend fun getMessages(): List<MessageDto>
10 }
```

Рисунок 4 – интерфейс MessageApi

DTO-модель содержит идентификатор пользователя, идентификатор сообщения, заголовок и текст.

Для локального хранения создана сущность MessageEntity, аннотированная @Entity. Таблица содержит поля, соответствующие данным сервера, а также флаг isLiked. Последний отвечает за состояние лайка и обновляется при нажатии на иконку.

```
1 package com.mobile.lab1.data
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "messages")
7 data class MessageEntity(
8     @PrimaryKey val id: Int,
9     val userId: Int,
10    val title: String,
11    val body: String,
12    val isLiked: Boolean = false
13 )
```

Рисунок 5 – сущность Room

Интерфейс MessageDao описывает методы доступа к таблице. В нем заданы запрос на получение всех сообщений, операция вставки списка с политикой REPLACE и метод обновления записи. Класс AppDatabase расширяет RoomDatabase и предоставляет DAO через абстрактное свойство.

Все решения по выбору источника данных инкапсулированы в MessageRepository. Репозиторий получает на вход API и DAO. При принудительном обновлении он вызывает сетевой запрос, преобразует полученные DTO в сущности и записывает их в базу. Затем список читается

из Room и возвращается вызывающему коду. Если во время запроса к серверу возникает ошибка сети, репозиторий вместо этого выдаёт содержимое локальной таблицы.

3.4 Лабораторная работа №4. Интерфейс, лайки, фоновые задачи и уведомления

Четвертая лабораторная работа направлена на улучшение пользовательского интерфейса и реализацию фоновых механизмов.

Список сообщений теперь отображается с помощью кастомного адаптера `MessageAdapter`. Каждый элемент представляет собой `MaterialCardView`, внутри которой расположены аватар пользователя, имя, текст сообщения и иконка лайка. В макете элемента применяются компоненты `Material Design`, что обеспечивает единый стиль в светлой и тёмной темах.

В адаптер передается обработчик нажатия на иконку лайка. При клике вызывается функция, которая меняет значение поля `isLiked` и передает обновленную модель обратно в `ViewModel`, а затем через репозиторий в базу `Room`.

```

9      class MessageAdapter(
10
11
12
13
14
15
16
17
18
19
20      4 Usages
21      inner class MessageViewHolder(
22          private val binding: ItemMessageBinding
23      ) : RecyclerView.ViewHolder( itemView = binding.root) {
24
25          1 Usage
26          fun bind(item: Message) {
27              binding.tvAuthor.text = item.authorName
28              binding.tvBody.text = item.body
29
30              val iconRes = if (item.isLiked) {
31                  R.drawable.ic_favorite_24
32              } else {
33                  R.drawable.ic_favorite_border_24
34              }
35              binding.ivLike.setImageResource(iconRes)
36
37              binding.ivLike.setOnClickListener {
38                  onLikeClick(item)
39              }
40          }
41
42      override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MessageViewHolder {
43          val inflater = LayoutInflater.from(parent.context)
44          val binding = ItemMessageBinding.inflate(inflater, parent, attachToParent = false)
45          return MessageViewHolder(binding)
46      }
47
48      override fun onBindViewHolder(holder: MessageViewHolder, position: Int) {
49          holder.bind(items[position])
50      }
51
52      override fun getItemCount(): Int = items.size
53  }

```

Рисунок 6 – обработка лайка в MessageAdapter

Для ручного обновления ленты на экране FeedFragment размещена плавающая кнопка FloatingActionButton. Нажатие на нее вызывает метод refresh во FeedViewModel, который обращается к репозиторию. Во время загрузки отображается ProgressBar, а возможные ошибки показываются в текстовом поле tvError.

Помимо ручного обновления, реализована фоновая синхронизация сообщений. Для этого создан класс SyncMessagesWorker, унаследованный от CoroutineWorker. В методе doWork воркер получает экземпляр базы данных AppDatabase, создаёт репозиторий MessageRepository и выполняет загрузку сообщений. В случае успешного выполнения сначала обновляются данные в

локальной базе, а затем вызывается приватный метод `showNotification`, который отвечает за формирование и отображение системного уведомления. Если во время работы возникает исключение (например, ошибка сети), метод возвращает `Result.retry`, что позволяет `WorkManager` повторить задачу позднее.

```
11 class SyncMessagesWorker(  
12     applicationContext: Context,  
13     params: WorkerParameters  
14 ) : CoroutineWorker(applicationContext, params) {  
15  
16     override suspend fun doWork(): Result {  
17         val db = AppDatabase.getInstance(applicationContext)  
18         val repo = MessageRepository(  
19             api = NetworkModule.api,  
20             dao = db.messageDao()  
21         )  
22  
23         return try {  
24             repo.loadMessages(forceRefresh = true)  
25             showNotification()  
26             Result.success()  
27         } catch (e: Exception) {  
28             Result.retry()  
29         }  
30     }  
31  
32     1 Usage  
33     private fun showNotification() {  
34         val builder = NotificationCompat.Builder(  
35             applicationContext,  
36             channelId = MessengerApp.CHANNEL_ID_SYNC  
37         )  
38         .setSmallIcon(R.drawable.ic_notifications_24)  
39         .setContentTitle("Messenger")  
40         .setContentText("Новые данные получены")  
41         .setPriority(NotificationCompat.PRIORITY_DEFAULT)  
42  
43         NotificationManagerCompat.from(applicationContext)  
44             .notify(id = 1001, notification = builder.build())  
45     }  
46 }
```

Рисунок 7 – фрагмент класса `SyncMessagesWorker`

Канал уведомлений с идентификатором `CHANNEL_ID_SYNC`

создается в классе `MessengerApp` при запуске приложения. В методе `showNotification` формируется объект `NotificationCompat.Builder` с небольшой иконкой, заголовком «Messenger» и текстом. Затем уведомление передается менеджеру уведомлений `NotificationManagerCompat`, который отображает его в системной шторке.

На устройствах с Android 13 и выше для показа уведомлений необходимо отдельное разрешение `POST_NOTIFICATIONS`. В рамках курсового проекта запрос этого разрешения реализуется в активити при первом запуске приложения. После того как пользователь подтверждает разрешение, фоновый воркер может показывать уведомления о том, что новые данные успешно получены.

4 АРХИТЕКТУРНЫЕ ОСОБЕННОСТИ

4.1 Структура пакетов

Код проекта организован по слоям. В пакете `data` сосредоточены классы, связанные с данными: API-интерфейс, сущности для Room, DAO, база данных и репозиторий. Пакет `viewmodel` содержит классы `FeedViewModel`, `ProfileViewModel` и `SettingsViewModel`, каждый из которых отвечает за состояние своего экрана. В корневом пакете находятся фрагменты, главная активность, адаптер списка сообщений, базовый класс `LoggingFragment`, а также классы, связанные с `WorkManager` и уведомлениями.

Такое разделение приближает структуру к классической схеме MVVM и упрощает сопровождение. Логика слоя данных не смешивается с кодом пользовательского интерфейса, поэтому добавление новых источников данных или переработка базы не требует изменения фрагментов.

4.2 Единый источник данных и офлайн-режим

Все данные, которые отображаются в ленте, поступают из локальной базы Room. Даже если сообщения приходят из сети, они сначала сохраняются в таблицу, а затем читаются из неё. В результате интерфейс

всегда опирается на один согласованный источник, что соответствует принципу единого источника правды. При отсутствии подключения к интернету репозиторий не делает сетевой запрос, а возвращает список сообщений из Room. Пользователь продолжает видеть последнюю загруженную информацию, а приложение корректно реагирует на потерю сети.

5 ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

Интерфейс приложения выполнен на основе компонентов Material Design и поддерживает два режима оформления: светлую и темную тему. Переключение темы влияет на все экраны приложения и демонстрирует использование DayNight темы в сочетании с ViewModel, которая хранит состояние выбранного режима

5.1 Лента сообщений

Главный экран приложения представляет собой ленту сообщений. В верхней части экрана расположен элемент MaterialToolbar с заголовком «Лента». Ниже находится список сообщений, реализованный с помощью RecyclerView. Каждый элемент списка оформлен как карточка MaterialCardView: слева отображается иконка пользователя, рядом выводятся условное имя отправителя и текст сообщения, справа размещена иконка лайка в виде сердечка. В правом нижнем углу экрана находится плавающая кнопка FloatingActionButton для ручного обновления данных.

Во время загрузки данных над списком показывается индикатор ProgressBar, при возникновении ошибок в отдельном текстовом поле выводится краткое сообщение. Навигационная панель BottomNavigationView закреплена в нижней части активности и обеспечивает переход к профилю и настройкам.

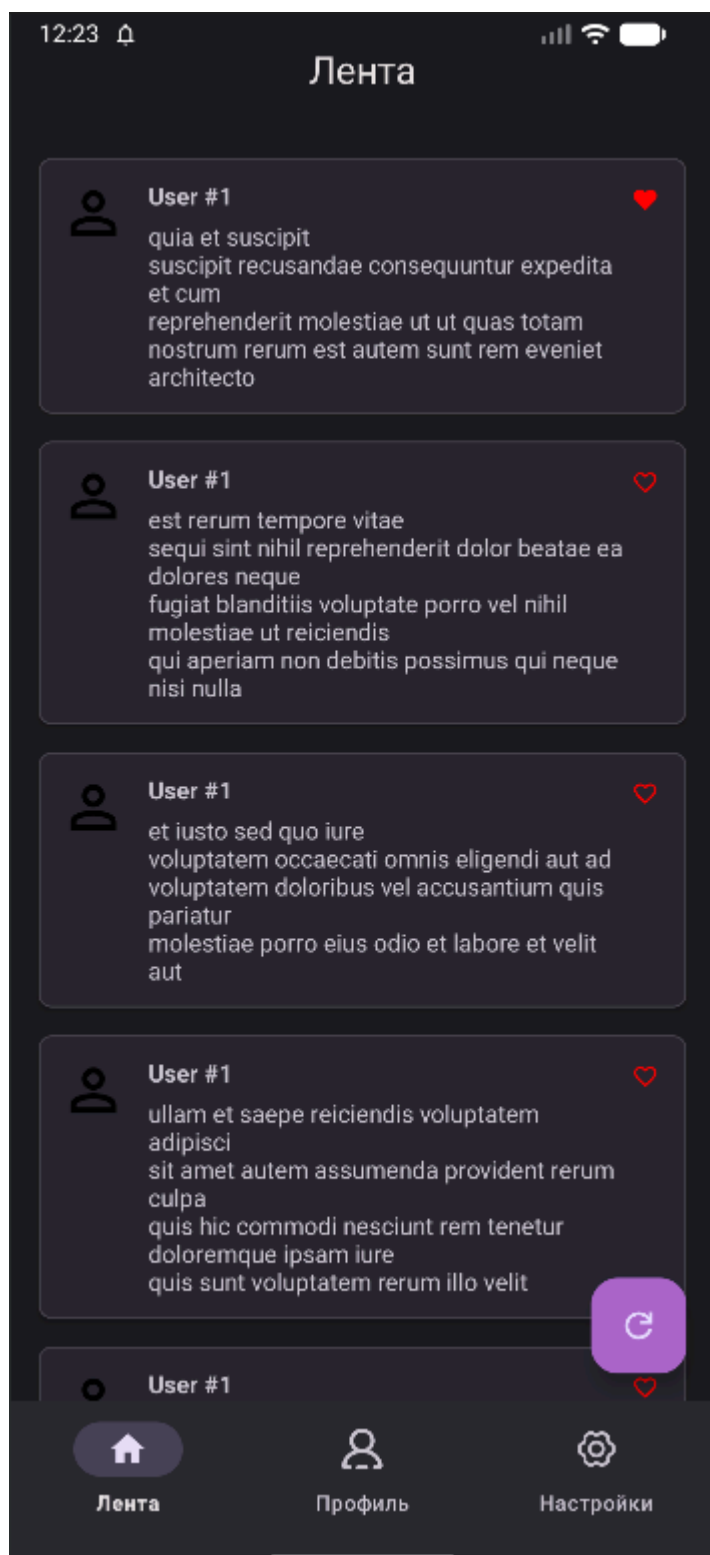


Рисунок 8 – Лента сообщений в темной теме

5.2 Профиль пользователя

Экран профиля реализован на основе вертикального `LinearLayout`. В верхней части размещен заголовок, далее идут два поля типа `EditText`: «Имя» и «Статус». Значения этих полей связаны с `LiveData` во `ProfileViewModel`. При открытии экрана туда подставляются данные, сохраненные ранее, а

любое изменение текста обновляет состояние ViewModel. Благодаря этому после поворота устройства введенные данные не пропадают.



Рисунок 9 – Экран профиля пользователя

5.3 Экран настроек

На экране настроек находится переключатель темной темы (MaterialSwitch или SwitchCompat в зависимости от реализации). При изменении его состояния вызывается метод ViewModel, который, в свою очередь, переключает режим DayNight во всём приложении. При желании

настройку можно дополнительно сохранять в SharedPreferences, чтобы она восстанавливалась после перезапуска приложения.



Рисунок 10 – Экран настроек

5.4 Уведомления и разрешения

Система уведомлений использует стандартные механизмы Android. Вспомогательный класс создает канал с умеренным приоритетом и формирует уведомление при успешной фоновой синхронизации. На устройствах с Android 13 и выше при первом запуске приложения активность инициирует запрос разрешения на отправку уведомлений. Только после

согласия пользователя WorkManager сможет показывать сообщения в строке состояния.

6 ТЕСТИРОВАНИЕ

Работоспособность приложения проверена вручную на эмуляторе Android и реальном устройстве. Особое внимание уделялось нескольким аспектам. Во-первых, проверялась корректность навигации между вкладками нижнего меню. Все переходы выполнялись без сбоев, состояние каждого экрана сохранялось при возврате назад. Во-вторых, тестировалось изменение имени и статуса в профиле с последующим поворотом устройства. Данные оставались на месте, что подтверждает корректную работу ViewModel.

Отдельно проверялась работа с сетью. При наличии подключения список сообщений загружался с сервера, после чего приложение могло запускаться без интернета, используя локальные данные. Нажатие на иконку лайка изменяло состояние записи и визуальное отображение сердечка. Плавающая кнопка обновления корректно инициировала повторный запрос. Было подтверждено, что фоновый воркер WorkManager выполняет синхронизацию по расписанию и показывает системное уведомление после успешного обновления. На Android 13+ уведомления отображались только после того, как пользователь разрешил их в системном диалоге.

В ходе тестирования критических ошибок, приводящих к падению приложения, выявлено не было. Отдельные сетевые таймауты обрабатывались и отображались пользователю в виде краткого сообщения об ошибке.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы создано учебное Android-приложение, демонстрирующее применение современных подходов к мобильной разработке. В приложении реализованы три основных экрана, единая навигация, архитектура MVVM, загрузка данных из REST API, кэширование в локальной базе Room, фоновая синхронизация через

WorkManager и система уведомлений. Интерфейс оформлен на базе компонентов Material Design и поддерживает переключаемую тему.

Построенная архитектура упрощает дальнейшее развитие проекта. При необходимости можно добавить авторизацию, реальную отправку сообщений, поиск по ленте, пагинацию и пуш-уведомления через Firebase Cloud Messaging. Кроме того, возможна интеграция модульного тестирования ViewModel и репозитория, а также настройка непрерывной интеграции.